MyScoutee
OFFICIAL PRODUCT DOCUMENTATION

LABORATORY OPERATIONS

Laboratory Access Manual

Where services run, how to sign in, and how the versioned GitLab laboratory connects builds, infrastructure and deployments.

DOCUMENT ID MSC-LAB-001	**DOCUMENT VERSION** 1.0.0
APPLICABLE SOURCE BASELINE MyScoutee 1.5.0 / MSC-129	**STATUS** ● Released
RELEASE DATE 2026-10-26	**LANGUAGE** EN

Document Control

Owner / audience	MyScoutee project / local laboratory operators
Document version	1.0.0, released 2026-10-26
Scope	MSC-129 GitLab lab, main backend and template. This is an access and workflow reference, not a production installation certificate.
Security boundary	No password, token or private key is included. Paths identify the owning local configuration. A fresh clone needs the operator's own credentials.

Table of Contents

Service addresses and ports 3

Sign-in and credential locations 4

Build, deployment and release controls 5

Infrastructure, registries and Kubernetes 6

Verification and accepted results 7

Configuration owners and reproduction 8

Read addresses as configured endpoints

`localhost` means the developer host running GitLab and the tunnels. An address only responds while its service or forwarding process is running. Run the application targets sequentially on this host; starting two application stacks together is not part of this laboratory workflow.

Snapshot at acceptance: template VMware is running; template monolith and all KVM test guests are stopped with data retained. The backend service inside VMware is stopped. Later operator actions can change this state.

1. Service Addresses and Ports

Service	Local address / port	Versioned setting owner
GitLab / group	http://localhost:8929 /myscoutee	CI `.env.example` `GITLAB_HTTP_PORT`
Git SSH	`localhost:2224`	`GITLAB_SSH_PORT`
Container Registry	`localhost:5050` Docker endpoint, not a web dashboard	`GITLAB_REGISTRY_PORT`
Backend monolith	https://localhost:18443 HTTP 18081; STUN 13479	Backend `config/ci/monolith.json`
Template monolith	https://localhost:18444 HTTP 18082; STUN 13480	Template `config/ci/monolith.json`
Backend KVM	http://localhost:18029	Backend `config/ci/kvm.json`
Template KVM	http://localhost:18030	Template `config/ci/kvm.json`
Backend VMware	https://localhost:8443 HTTP 8080	Backend `config/ci/variables.json`
Template VMware	https://localhost:18445 HTTP 18083	Template `config/ci/variables.json`
Shared SonarQube	http://localhost:9000	SonarQube `server.json`
KVM Grafana	Backend: `http://localhost:13029` Template: `http://localhost:13030`	Product `config/ci/kvm.json`; requires monitoring startup
KVM Prometheus	Backend: `http://localhost:19029` Template: `http://localhost:19030`	Same monitoring configuration

These are the CI target defaults, not the separate native-development ports. Environment overrides are mapped in each product's `config/ci/variables.json` from the CI repository's ignored `.env`.

VMware: open the HTTPS address directly. Guest port 80 may redirect to HTTPS port 443 without preserving a custom host tunnel port. The versioned forwarding helper maps host ports to guest ports; it does not rewrite application redirects.

2. Sign-in and Credential Locations

Access	Where to find the local input
GitLab administrator	User `root`. Generated initial password: CI `.local/state/root-password`, or the equivalent location under `LAB_STATE_DIR`. If changed through GitLab, use the current operator-managed password.
GitLab API / runner	CI ignored state and provisioning scripts. `scripts/local_gitlab.py` resolves the API credential. Runner tokens are managed by setup; they are not application login passwords.
GitHub / npm / Gradle / Docker	CI `.env` contains file locators such as `GITHUB_TOKEN_FILE`, `LAB_NPMRC_FILE`, `LAB_GRADLE_PROPERTIES_FILE` and `LAB_DOCKER_CONFIG_FILE`. Follow each locator locally. Do not paste file contents into logs.
VMware SSH and TLS	Each product owns `config/packaging/client-tools/vmware.local.env` and ignored `.local/private/vmware/`: SSH key, known hosts, CA certificate and release public key. The target/user come from the local install configuration.
Release signing	Each product's ignored `.local/private/config/release/`; CI receives the configured signing-key FILE variable. A public verification key cannot replace a private signing key.
Installed application Operator	On the guest: `/etc/<project>/<project>.env` and `<project>.secrets.env`. See `OPERATOR_USER`, `OPERATOR_BOOTSTRAP_EMAIL` and the password hash. The initial plaintext password is emitted when generated; it cannot be recovered from its hash.
SonarQube	SonarQube repo `.state/myscoutee/.env`. `SONAR_ADMIN_PASSWORD` is an optional private record of the password already set in the UI; `SONAR_TOKEN` authenticates scans. The database password is not the UI password.
Grafana	The selected product's effective runtime `lab.grafanaPassword` and generated monitoring configuration. Existing lab defaults are local-only. Use the deployment's actual value, not a credential copied from another product.

`<project>` is `myscoutee` or `myscoutee-template`. Ordinary application users use the configured application sign-in method; a GitLab or SSH account grants no application role.

Private values belong to their owning repository's ignored local files. Public defaults, file-variable mappings and helper scripts belong in Git. Check a local file with `git check-ignore -v PATH` before storing a secret there. Never publish credential files as CI artifacts.

3. Build, Deployment and Release Controls

Open the product in GitLab, then **Build > Pipelines** and select a pipeline. The graph belongs to a pipeline run, not to the repository overview. Job names describe the operation; the play button runs a manual job.

Operation	Expected behavior
Normal build	Automatic after a qualifying source push or dependency trigger. Feature branches can build and be tested. A merge rebuilds master.
Tests	Optional manual jobs. Skipping them does not certify that tests passed.
`package:kvm` `package:deb`	Manual resource-intensive assembly. KVM uses the product's domain groups and reuses unchanged images. A full DEB is a separate deployment artifact, not an implicit part of every release.
`infra:start`	Optional manual preparation/startup. Checks current assets and infrastructure first. An existing target does not require clicking it again.
`deploy:monolith` `deploy:kvm` `deploy:vmware`	Manual, target-specific deployment. Can use an unpublished candidate. Checks actual prerequisites; it does not require infra to have run in this pipeline.
`deploy:agent`	Installs/configures the GitLab Kubernetes agent. This is a deployment operation, not simply an application health check.
`verify:*`	Checks the selected deployed target and records passed, failed and skipped checks. Read the job report before treating the target as ready.
Release / publish	Manual integrated version change, build/publication, source tag and GitHub release. Release work uses a release branch and integration back to master; feature-branch test deployment is not a release.

Runners and concurrency

Two runner registrations have different roles: isolated validation and trusted native builds. The native runner can execute **four concurrent build jobs**; four visible runner entries are not required. Target-mutating jobs use resource locks. Four slots do not authorize running both application environments together on this host.

GitHub Actions boundary

The newly supplied conversion stays in `config/ci/github-actions.yml`, outside the active workflow directory. It is a presentation conversion, not a qualified runtime. Existing GitHub workflows keep their prior behavior. Pages publication remains manual.

4. Infrastructure, Registries and Kubernetes

GitLab area	What it contains
Package Registry	Generic infrastructure inputs such as Ubuntu cloud image, Terraform executable/provider and k3s binary/offline images, plus versioned release packages. A candidate build normally stays in job artifacts/local cache.
Container Registry	Versioned released application images. Build candidates can be reused from the native Docker cache. The release adapter publishes configured release images to GitLab and GitHub's container registry (GHCR).
Terraform Module Registry	The small reusable `labkvm/libvirt` module owned by `myscoutee-microservices`. It is not the Ubuntu installer or VM disk. Module publication is automatic; its version is independent of an application candidate version.
Terraform state	Private deployment state in the product's GitLab project when the GitLab backend is selected. Backend and template use separate state even when both state names are `kvm`.
Kubernetes / Environments	Agent connection and the configured environment dashboard. The agent is declared in the CI repository; product configuration supplies context and namespace. It does not create a second application runtime.

Registry-first infrastructure preparation

The manual infrastructure flow checks the registry for the pinned assets. Missing inputs can be downloaded, checksum-verified and uploaded for reuse. Existing registry entries avoid repeating upstream multi-gigabyte downloads. On another machine, the required bytes still have to move from the registry to that machine.

The separate `scripts/import-infra-assets.py` operator tool can seed the registry from an already downloaded local file. Its source is committed. It supports the initial cache population; deployment does not silently replace this with an upstream download.

One shared module, independent product configuration

The current implementation has one reusable Terraform module with two independent consumers, rather than two duplicated modules. Each product owns `config/ci/kvm.json` and `config/microservices/deploy/runtime.json`, its VM identity, state and composition. The backend native memory setting is 8192 MiB; the template has its own setting. CI does not reintroduce the removed 5 GiB override.

Kubernetes dashboard

In the product, open **Operate > Environments** and the KVM environment. The configured agent/context supplies access; no per-visit server URL entry is needed. Template's qualified dashboard is environment 7. It can show disconnected after the VM is stopped. The group Kubernetes page lists agents; it is not the installer registry.

5. Verification and Accepted Results

The operator accepted all three template targets on 26 October 2026. Each target was reviewed before the next one started. These are recorded runs, not a claim that every optional test or every future deployment has passed.

Target	Recorded GitLab evidence	Result
Template monolith	Pipeline 563 Build 2201 Deploy 2210 Verify 2215	14 passed, 0 failed, 0 skipped. Browser accepted. Stopped; volumes retained.
Template KVM	Pipeline 567 Package 2228; infra 2231 Deploy 2232; verify 2237 Agent retry 2241	10 public checks passed. All four domain images reused. Agent connected; browser accepted. Stopped; data retained.
Template VMware	Pipeline 571 Build 2245 Deploy 2253 Verify 2258	11 passed, 0 failed, 3 skipped. Three HTTP checks skipped with the custom HTTPS tunnel. Browser accepted; left running.

What readiness means

The deployed runtime was checked beyond the web server returning a page: application runtime readiness, API health and the applicable public asset/localization/document checks. A page loading alone is insufficient. The VMware result also confirmed `/api/runtime-status` with `ready: true` and `known: true`.

Recorded release and packaging exercises

SonarQube pipeline 488 / release job 2159 exercised the integrated release path. Backend pipeline 489 / job 2165 exercised DEB packaging and actual Debian metadata inspection. Trial publications were withdrawn; source/package baseline remains 1.5.0. Qualification candidate directories may carry an internal next-patch version without constituting a public release.

Boundaries

No Netcup deployment is part of MSC-129. The newly added GitHub Actions conversion was not executed. No new backend or DEB rerun was required for this final documentation. Monitoring ports are documented configuration, not an independent monitoring qualification.

If the application appears offline

1. Confirm the intended target is running and the correct HTTPS/HTTP port is selected.
2. Read that target's latest deploy and verify job output.
3. Check runtime readiness and the target's service/pod health before changing source or restarting unrelated targets.
4. For a stopped KVM, manually run infra if needed. For a missing installer, use manual asset preparation; do not hide it inside application deployment.

6. Configuration Owners and Reproduction

Repository / path	Purpose
CI `.env.example`, `compose.yaml`	GitLab service, port and runner defaults; local overrides in ignored `.env`.
CI `ci/projects.json`, `examples/variables.json`	Project/dependency catalog, source pins and variable declarations.
Product `config/ci/`	Pipeline entry, release name, deploy targets, public ports and private input mappings.
CI `scripts/configure.py`	Applies declared GitLab variables and library pins through the API.
CI `scripts/runner-host.py`, `guides/development/ci/runner-host.md`	Dedicated host account and scoped provisioning; example in `examples/`, actual input in ignored `.local/private/runner-host.env`.
CI `scripts/remote.sh`, `deploy.sh`, `verify-remote.sh`	Existing remote deployment and verification flow.
Packaging `client-tools/forward-localhost.sh`, `remote-transport.sh`	Versioned forwarding and SSH transport. Product `config/packaging/client-tools/manifest.json` selects the shipped tools.
CI `ci/retention.json`, `scripts/retention.py`	Lab retention/cleanup policy. Maintenance is separate from the application deployment pipeline.
CI `guides/development/ci/verification.md`, `guides/development/ci/MSC-129-plan.md`	Detailed evidence and final task checklist.

Reproduce configuration

From the CI checkout, after supplying its local inputs:

```
bash scripts/lab.sh init
python3 scripts/setup.py
# Later, preview and reconcile declared variables:
python3 scripts/configure.py
python3 scripts/configure.py --apply
```

Setup mutates the local GitLab configuration and imports source; it does not publish an application release or deploy a target. Refer to the CI README for prerequisites and the dedicated host-account setup.

Rebuild this document

From this backend or template checkout, using the installed frontend documentation dependencies and local Chrome/Chromium:

```
node guides/publication/manuals/build-manuals.mjs MSC-LAB-001
```

The catalog and HTML source live beside this PDF. This laboratory manual is maintained in the backend and template guides; both catalog entries disable copying to their frontend repositories. Keep the document version, cover, catalog and links aligned when changing the manual.

CI source and operating instructions · MSC-129 task